Making an algorithm for solving linear equation systems

Timothée Monin

April 2025

Abstract

This document shows some personal research done on my own for finding an algorithm to solve any system of n linear equations with n unknowns, based on the coefficients and constant terms of the equations in the system. We will first treat the base case $n = 2$, and then we will extend it to the general case.

Contents

1 The base case: linear systems of two equations with two unknowns

First of all, we will study the base case of systems of two linear equations with two unknowns (case $n = 2$). We could also consider that the base case is the case $n = 1$, but this case is trivial to solve.

So, that said, let us consider the following system of equations:

$$\begin{cases} a_1 x + b_1 y = c_1 \\ a_2 x + b_2 y = c_2 \end{cases}$$

where the unknowns are x and y, and the values a_1, a_2, b_1 and b_2 are the coefficients of the system.

1.1 Finding the solutions

We first find y in terms of x (it could be in the other direction, this choice is arbitrary). For example, if we use the first equation of the system:

$$\begin{aligned} & a_1 x + b_1 y = c_1 \\ \iff & b_1 y = c_1 - a_1 x \\ \iff & y = \frac{c_1 - a_1 x}{b_1} \text{ for } b_1 \neq 0 \end{aligned}$$

This only works for $b_1 \neq 0$, but if we choose the other equation, we instead have:

$$y = \frac{c_2 - a_2 x}{b_2}$$

which only works for $b_2 \neq 0$.

Note that at this point, we have two different solutions for expressing y in terms of x, with two different restrictions about the coefficient values.

Also note that the two expressions only differ by the number/index of the equation of the coefficients (for example a_1 is replaced by a_2 in the second relation).

Now we can arbitrarily choose one of the relations, to express y in terms of x in the equation which was not used to find to find the relation, to finally find the solution for x. We have, using the first expression of y in terms of x:

$$
\begin{aligned}
& a_2x + b_2y = c_2 \\
\iff & a_2x + b_2(\frac{c_1 - a_1x}{b_1}) = c_2 \text{ for } b_1 \neq 0 \\
\iff & a_2x + \frac{b_2c_1 - a_1b_2x}{b_1} = c_2 \\
\iff & a_2b_1x + b_2c_1 - a_1b_2x = b_1c_2 \\
\iff & a_2b_1x - a_1b_2x = b_1c_2 - b_2c_1 \\
\iff & a_1a_2b_1b_2(\frac{1}{a_1b_2}x - \frac{1}{a_2b_1}x) = b_1c_2 - b_2c_1 \text{ for } a_1, a_2, b_2 \text{ (and } b_1) \neq 0 \\
\iff & \frac{1}{a_1b_2}x - \frac{1}{a_2b_1}x = \frac{b_1c_2 - b_2c_1}{a_1a_2b_1b_2} \\
\iff & (\frac{1}{a_1b_2} - \frac{1}{a_2b_1})x = \frac{b_1c_2 - b_2c_1}{a_1a_2b_1b_2} \\
\iff & (\frac{a_2b_1 - a_1b_2}{a_1a_2b_1b_2})x = \frac{b_1c_2 - b_2c_1}{a_1a_2b_1b_2} \\
\iff & x = \frac{b_1c_2 - b_2c_1}{a_1a_2b_1b_2}\frac{a_1a_2b_1b_2}{a_2b_1 - a_1b_2} \text{ for } a_2b_1 - a_1b_2 \neq 0 \\
\iff & x = \frac{b_1c_2 - b_2c_1}{a_2b_1 - a_1b_2} \text{ for } a_1, a_2, b_1, b_2, \text{ and } a_2b_1 - a_1b_2 \neq 0
\end{aligned}
$$

1.2 The special cases

So we have found an - almost systematic - solution for x. Now, let us study these different cases for the forbidden values of the solution:

- **If any of $\mathbf{a_1}, \mathbf{a_2}, \mathbf{b_1}$ or $\mathbf{b_2} = 0$**

  In this case, either the system will have solution for x and y like normal, or it will admit only a solution for x or y, or it will have no solution. But we won't focus on this case and we will discard it.

- **If $\mathbf{a_2b_1 - a_1b_2} = 0$**, but a_1, a_2, b_1, and $b_2 \neq 0$

  In this case, we have $\frac{a_1}{a_2} = \frac{b_1}{b_2}$, since $a_2b_1 - a_1b_2 = 0 \iff a_1b_2 = b_1a_2 \iff \frac{a_1}{a_2} = \frac{b_1}{b_2}$, and we already assumed that a_1, a_2, b_1, and $b_2 \neq 0$.

Let $k = \frac{a_1}{a_2} = \frac{b_1}{b_2}$. We have $a_1 = ka_2$ and $b_1 = kb_2$.

It means that the left-hand parts of the two equations of the system are proportional, because $a_1x + b_1y = ka_2x + kb_2y = k(a_2x + b_2y)$.

We then distinguish two cases:

- **If $\mathbf{c_1 = kc_2}$**
  In this case, we can get the first equation by multiplying both sides of the second equation by k. This means that $a_2x + b_2y = c_2 \iff a_1x + b_1y = c_1$, so we only really have one equation, with two unknowns.
  But one linear equation with two unknowns and non-zero coefficients has infinitely many solutions. Proof:
  Let us consider the equation $ax + bx = c$, with non-zero coefficients. For any solution for x, there exists a solution for y and it is:
  $$y = \frac{c - ax}{b} \text{ for } b \neq 0$$
  There are infinitely many values possible for x, so therefore there are infinitely many solutions that work for x and y.
  We can now conclude that in the current case, there will be infinitely many solutions for the equation system.
- **If $\mathbf{c_1 \neq kc_2}$**
  In this case we have:

$$\begin{aligned} a_2x + b_2y = c_2 &\iff ka_2x + kb_2y = kc_2 \\ &\iff a_1x + b_1y = kc_2 \\ &\implies a_1x + b_1y \neq c_1 \end{aligned}$$

  This means that if we find a solution for x and y that satisfies the second equation, then this solution will not work with the first equation. Therefore, the system of equations admits no solutions in the current case we are dealing with.

1.3 Making the (almost) general algorithm

We have finished treating the special cases, which correspond to the forbidden values found for the solution for x. But now, how we can know y ? Well,

we can just mirror the solution previously found. We can use the solution for x and use it for y; we just have to modify the coefficient names in the solution: a coefficients become b coefficients, and vice versa.

We are now ready to find an algorithm to solve a system of two linear equations with two unknowns. But because the case where any of a_1, a_2, b_1 or $b_2 = 0$ complexifies the algorithm, we will omit it.

Input : The coefficients and constant terms of the equation, but with a_1, a_2, b_1 and $b_2 \neq 0$
Output: The solutions for x and y

if $a_2b_1 - a_1b_2 = 0$ **then**
  if $\frac{a_1}{a_2} = \frac{c_1}{c_2}$ **then**
    return *"Infinitely many solutions"*
  else
    return *"No solutions"*
  end
else
  $x \leftarrow \frac{b_1c_2 - b_2c_1}{a_2b_1 - a_1b_2}$;
  $y \leftarrow \frac{a_1c_2 - a_2c_1}{b_2a_1 - b_1a_2}$;
  return x, y
end

Algorithm 1: An algorithm to find the solutions x and y of a linear system of two equations with non-zero variable coefficients

2 Extending to the (almost) general case

We will now consider the general case, i.e. systems of n equations with n unknowns:

$$\begin{cases} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n = b_1 \\ a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n = b_2 \\ \vdots \\ a_{n1}x_1 + a_{n2}x_2 + \cdots + a_{nn}x_n = b_n \end{cases}$$

where the unknowns are $x_1, x_2, \ldots, x_n$, the coefficients are the a terms, and the *constant terms* are the b terms.

Note that we must have $n \geq 2$ (with $n = 1$, it is not a *system* of equations any more).

Because the solution we found for the case $n = 2$ was restricted to non-zero coefficients, we will only study equation systems with non-zero coefficients.

2.1 Reducing an equation system to a base case

In order to find a general solution, one idea could be to "eliminate" one unknown at a time, and this way convert repeatedly an n equations linear system to a $n - 1$ equations linear system, until we reach the case $n = 2$. If we want to do that, we first have to find an algorithm that does this elimination process.

Let us assume we want to eliminate the last unknown of the system, i.e. x_n. We will proceed by finding its expression in terms of the other unknowns, from one of the equations. So, if we take the m-th equation (with $m \leq n$ obviously), we will have:

$$\begin{aligned}
& a_{m1}x_1 + a_{m2}x_2 + \cdots + a_{mn}x_n = b_m \\
\iff & a_{mn}x_n = b_m - a_{m1}x_1 - a_{m2}x_2 - \cdots - a_{m-1n}x_{m-1} \\
\iff & x_n = \frac{b_m - a_{m1}x_1 - a_{m2}x_2 - \cdots - a_{mn-1}x_{n-1}}{a_{mn}} \text{ because } a_{mn} \neq 0 \\
\iff & x_n = -\frac{a_{m1}}{a_{mn}}x_1 - \frac{a_{m2}}{a_{mn}}x_2 - \ldots - \frac{a_{mn-1}}{a_{mn}}x_{n-1} + \frac{b_m}{a_{mn}}
\end{aligned}$$

So we know how to express the last unknown of an equation in terms of the other unknowns (plus a constant term). To eliminate the last variable in one of the equations, we have to pick another equation and calculate the last variable in terms of the others, and then plug in the expression found into the first equation. To plug in the expression found into the equation, we first multiply the coefficients (and the constant term) of the expression by the coefficient of the last unknown in the equation. Then, for each unknown in the equation we add to its coefficient the corresponding coefficient in the expression (because coefficients are in the left-hand side). Finally, we subtract the constant term of the expression to the constant term in the right-hand part of the equation (because constant terms are in the right-hand side).

To express it mathematically, if we want to eliminate the last unknown of the k-th equation of the system using the m-th equation, we will apply the following relation:

$$
\begin{aligned}
& a_{k1}x_1 + a_{k2}x_2 + \cdots + a_{kn}x_n = b_k \\
\iff & a_{k1}x_1 + a_{k2}x_2 + \cdots + a_{kn}(-\frac{a_{m1}}{a_{mn}}x_1 - \frac{a_{m2}}{a_{mn}}x_2 - ... - \frac{a_{mn-1}}{a_{mn}}x_{n-1} + \frac{b_m}{a_{mn}}) = b_k \\
\iff & (a_{k1} + (-\frac{a_{m1}}{a_{mn}}) \cdot a_{kn}) \cdot x_1 + \cdots + (a_{kn-1} + (-\frac{a_{mn-1}}{a_{mn}}) \cdot a_{kn}) \cdot x_{n-1} = b_k - \frac{b_m}{a_{mn}} \cdot a_{kn}
\end{aligned}
$$

For our algorithm, we will take the first equation of the system to express the last unknown in terms of the others. Once we have replaced the last variable in the other equations, we can discard the first equation, because we now have $n-1$ unknowns, so we only need $n-1$ equations.

That explanation was a little bit unclear, but now let us make the algorithm to eliminate the last variable of the equation system:

Input : The (non-zero) coefficients and the constant terms of the system of n equations with n unknowns
Output: The $n-1$ system of linear equations
for $i = 2$ **to** n **do**
  for $j = 1$ **to** $n-1$ **do**
    $a_{ij} \leftarrow a_{ij} + (-\frac{a_{1j}}{a_{1n}}) \cdot a_{in}$
  end
  $b_i \leftarrow b_i - (\frac{b_1}{a_{1n}}) \cdot a_{in}$;
  remove a_{in}
end
remove $a_{1...}$ coefficients;
return *all defined coefficients and constant terms*

Algorithm 2: An algorithm to eliminate the last unknown of an linear equation system with non-zero coefficients

2.2 Making the full algorithm

Now that we have an algorithm eliminate an unknown of a system with non-zero coefficients to produce a system of $n-1$ equations with $n-1$ unknowns,

we can start finding an algorithm to solve a linear system. In order to do that, let us take an example.

For example, let us take the case $n = 3$ (still with non-zero coefficients). In this case, we have 3 unknowns. We will first apply the **Algorithm 2** to eliminate the last unknown (x_3 here). We will now have a system with only 2 equations and 2 unknowns, which we know how to solve. Then, once we have found the values for x_1 and x_2, we can apply the formula found earlier which gives the value of x_n (here x_3) in terms of all the other values in a one specific equation in the system. In our algorithm, we will use the first equation to find a solution for the last unknown, but this choice is arbitrary. At this point, the system will be solved.

This process works for $n = 3$ but it also works for *any* size.

Now, we are ready to make the (quite informal) algorithm to solve a linear equation system with non-zero coefficients:

Input : The (non-zero) coefficients and the constant terms of the equation system
Output: The solutions of the system
if $n = 2$ **then**
  Use **Algorithm 1** to solve the system;
  return *the result(s) found*
else
  Use **Algorithm 2** to reduce the size of the system;
  Recursively call this algorithm to find the solutions of the reduced system;
  if *"Infinitely many solutions"* **then**
    return *"Infinitely many solutions"*
  else if *"No solutions"* **then**
    return *"No solutions"*
  end
  $x_n \leftarrow -\frac{a_{11}}{a_{1n}}x_1 - \frac{a_{12}}{a_{1n}}x_2 - ... - \frac{a_{1n-1}}{a_{1n}}x_{n-1} + \frac{b_1}{a_{1n}}$; `/* We take the first equation to find the solution for` x_n `*/`
  return *the solutions found*
end

Algorithm 3: An algorithm to find the solutions of a linear system of equations with non-zero coefficients

Note that we could have chosen the base case to be $n = 1$, and it would have made the **Algorithm 1** much simpler. But we can justify this choice

by the little bit of efficiency gained, and with the fact that the solution found for linear systems of 2 equations with 2 unknowns is quite simple and elegant.

3 But this only works with non-zero coefficients, doesn't it?

The algorithm found previously only works with non-zero coefficients. Another problem we did not talk about is the case where the **Algorithm 2** returns a reduced system with one or more null coefficients.

This means the algorithm found is incomplete as it cannot handle null coefficients. In order to make it complete, one idea could be to find algorithm to transform a system *with* some null coefficients to a system with all coefficients being non-zero.

3.1 Ensuring the coefficients are non-zero

The basic idea is to eliminate these null coefficients is to "add" equations together. For each null coefficient, we add the the left and right-hand sides of the equation containing the null coefficient with another equation of the system, and replace the original equation with the resulting equation. In this method we have to choose the equation to add to the first one to have a non-zero coefficient in front of the unknown which has the null coefficient in the first equation. If no such equation exists, it means that the system only really has less than n unknowns, and therefore it is invalid. After "removing" a null coefficient, we have to process the equation in which it was to ensure that another null coefficient wasn't created when adding the other equation.

To illustrate the idea, let us use it on an example:

$$\begin{cases} 0x - 3y = 6 \\ 8x + 0y = 2 \end{cases}$$

This system of equation, chosen at random, has two different null coefficients.

Here, we would first add the equation number 2 to the equation number 1. The system now becomes:

$$\begin{cases} 8x - 3y = 8 \\ 8x + 0y = 2 \end{cases}$$

The system still remains equivalent to the first one, since we can "subtract" the second equation to get the original equation number 1.

We can now eliminate the second null coefficient of the system, by adding the first equation to the second:

$$\begin{cases} 8x - 3y = 8 \\ 16x - 3y = 10 \end{cases}$$

Now, let us make the general algorithm:

Input : The original system to process
Output: An equivalent system with non-zero coefficients (if the given system only has non-zero coefficients, it remains unchanged)

$i \leftarrow 0$;
while $i \neq n$ **do**
  $j \leftarrow 0$;
  while $j \neq n$ **do**
    if $a_{ij} = 0$ **then**
      pick any number m for which $a_{mj} \neq 0$;
      for $k = 1$ **to** n **do**
        $a_{ik} \leftarrow a_{ik} + a_{mk}$; `/* Adding coefficients */`
      end
      $b_i \leftarrow b_i + b_m$; `/* Adding the constant term */`
      $j = 0$; `/* Checking the current equation again */`
    else
      $j \leftarrow j + 1$;
    end
    $i \leftarrow i + 1$;
  end
end
return *the coefficients and constant terms*

Algorithm 4: An algorithm to ensure a system of linear equations does not have null coefficients

3.2 The final solving algorithm

We can now make the general algorithm for solving *any* system of n equations with n unknowns, even if it does not have only non-zero coefficients.

Note that this final algorithm only is a little modification of the **Algorithm 3**.

Input : The coefficients and the constant terms of the equation system
Output: The solutions of the system
Ensure non-zero coefficients with the **Algorithm 4**;
if $n = 2$ **then**
    Use the **Algorithm 1** to solve the system;
    return *the result found*
else
    Use the **Algorithm 2** to reduce the size of the system;
    Recursively call this algorithm to find the solutions of the reduced system;
    if *"Infinitely many solutions"* **then**
        return *"Infinitely many solutions"*
    else if *"No solutions"* **then**
        return *"No solutions"*
    end
    Ensure non-zero coefficients with the **Algorithm 4**;
    $x_n \leftarrow -\frac{a_{11}}{a_{1n}}x_1 - \frac{a_{12}}{a_{1n}}x_2 - ... - \frac{a_{1n-1}}{a_{1n}}x_{n-1} + \frac{b_1}{a_{1n}}$;
    `/* We take the first equation to find the solution for` x_n `*/`
    return *the solutions found*
end

Algorithm 5: An algorithm to find the solutions of a linear system of n equations with n unknowns

4 Testing the algorithm

Now that the algorithm is finished, we can test it with an example. This cannot replace a formal proof, but it at least shows that one particular implementation works for some examples.

4.1 An implementation in Python

To test the algorithm, I made a quick implementation of it in Python. Here are the different algorithms (except the **Algorithm 3**, which is replaced by the **Algorithm 5**) in Python:

Algorithm 1:

```
import copy  # for making copies of lists (used for the algorithms 2 and 4)

def solve_system_two_unknowns(a_1, b_1, c_1, a_2, b_2, c_2):
    """Implementation of the Algorithm 1"""
    if a_2 * b_1 - a_1 * b_2 == 0:
        if a_1 * c_2 == c_1 * a_2: # Same as a_1 / a_2 == c_1 / c_2
            return "Infinitely many solutions"
        else:
            return "No solutions"

    else:
        x = (b_1 * c_2 - b_2 * c_1) / (a_2 * b_1 - a_1 * b_2)
        y = (a_1 * c_2 - a_2 * c_1) / (b_2 * a_1 - b_1 * a_2)
        return x, y
```

Algorithm 2:

```
def reduce_system(system):
    """Implementation of the Algorithm 2"""
    n = len(system)
    result = system.copy()
    for i in range(1, n):  # we work with zero-based indexing
        for j in range(0, n - 1):  # same
            result[i][j] += (-system[0][j]/system[0][n - 1]) * system[i][n - 1]
        result[i][-1] -= (system[0][-1] / system[0][n - 1]) * system[i][n - 1]
        del result[i][n - 1]
    del result[0]
    return result
```

Algorithm 4:

```python
def ensure_non_zero_coefs(system):
    """Implementation of the Algorithm 4"""
    result = system.copy()
    n = len(system)
    i = 0
    while i != n:
        j = 0
        while j != n:
            if result[i][j] == 0:
                for m in range(0, n):  # picking m
                    if result[m][j] != 0:
                        break

                for k in range(0, n):
                    result[i][k] += result[m][k]
                result[i][-1] += result[m][-1]

                j = 0

            else:
                j += 1
        i += 1
    return result
```

Algorithm 5:

```python
def solve_system(system):
    """Implementation of the Algorithm 5"""
    system = ensure_non_zero_coefs(system)
    n = len(system)
    if n == 2:
        return solve_system_two_unknowns(*system[0], *system
    [1])
    else:
        reduced_system = reduce_system(system)
        solutions_found = solve_system_non_zero_coefs(
    reduced_system)
        if not isinstance(solutions_found, tuple):
            # Infinitely many or no solutions
            return solutions_found

        system = ensure_non_zero_coefs(system)

        x_n = 0
        for m in range(0, n - 1):
            x_n -= (system[0][m] / system[0][n - 1]) *
    solutions_found[m
        x_n += system[0][-1] / system[0][n - 1]

        return solutions_found + (x_n,)
```

An equation system is represented as a list of the equations in the system. Each equation in the system is represented as a list containing in order its coefficients and its constant term. The complete file containing the algorithm is named *linear_system_solver.py*.

4.2 A first example

We will take as an example the following system:

$$\begin{cases} 2x + 3y - z = 4 \\ x - 2y + z - 2t = -3 \\ x + 9y + 2z - t = 8 \\ 3y - 3z + 3t = 9 \end{cases}$$

It has 4 linear equations with 4 unknowns (x, y, z and t). We can use the implementation of the **Algorithm 5** to solve it. If we evaluate *solve_system([[2, 3, -1, 0, 4], [1, -2, 1, -2, -3], [1, 9, 2, -1, 8], [0, 3, -3, 3, 9]])*, we get the tuple *(-8.0, 3.0, -11.0, -11.0)*, which corresponds to the respective solutions for the unknowns. It means the solution is $x = -8$, $y = 3$, $z = -11$ and $t = -11$.

This solution was confirmed by Wolfram Alpha. We can also verify it by plugging the solutions found into the equations, and see that it works.

4.3 A randomised test program

I also implemented a randomised test program in Python that randomly generates a given number of linear systems up to a given size, and tests the solutions found by the solving program using the *sympy* library. It also uses the *fractions* module of the Python standard library to manipulate fractions, and thus have exact results. I tested thousands of systems with sizes sometimes up to 100, and I never ran into an issue. To execute the test program, run the file *linear_systems_tests.py* with Python (having installed the *sympy* library). To modify the values of the test, just change the default values of the function *test_random_systems()*.

5 Conclusion

We have found an algorithm to solve a linear system of equations (with the **Algorithm 5**). This algorithm was not formally proven, but it was at least justified and extensively tested with a Python program. There exist

many other (probably better) ways of solving linear systems (like Gaussian elimination which is very close to the algorithm I made), but I am 15 years old, and the goal was to make an algorithm on my own, which is what I did. Another issue with my algorithm is that it is not very efficient; there exist much more efficient methods.

Important note: I noticed that a special case was not treated, and not included in the solving algorithm. It is the case where the "reducing algorithm" (**Algorithm 2**) returns a system with less unknowns than equations, i.e. there is an unknown x_m for which $\forall k \leq n, a_{km} = 0$. Even if this case is *very* rare, it was not treated and thus the solving algorithm found is not fully complete.